Appendix A – Master Project Cover Page

PROJECT TITLE: **"Subsampling vs Ridge Regularization vs Lasso Regularization vs Elastic Net"**

By: Pranav Senthilkumaran (RUID: 238000129), Master's in Data Science, Rutgers University

Supervising Professor: Professor Han Xiao

Professor Signature: Xiao Han

Date Signed: 04-17-2025

A Research Project Submitted in
Partial Fulfillment of the
Requirements for the
Degree of Master of
Science in Statistics
Specialization in Data Science for **MSDS**
At Rutgers, the State University of New Jersey

Graduation Date: May 2026

Abstract

High-dimensional regression models are vulnerable to **overfitting** due to **multicollinearity** and noise, making regularization techniques essential for reliable performance. This project investigated the theoretical and empirical relationship between **Ridge Regression (ℓ2 Regularization)**, **Lasso Regression (ℓ1 Regularization)**, **Elastic Net (Convex Combination of ℓ1 + ℓ2 Regularization)**, and **Subsampling-based Ensemble Linear Models**. Using controlled **high-dimensional simulations** with n=200 observations and p=100 predictors, the study compared model behavior under different regularization strategies.

The analysis evaluated **Test Mean Squared Error (MSE)**, performed an empirical **Bias–Variance Decomposition**, examined **Model Stability**, analyzed **Sensitivity to Subsampling Ratios**, and studied **Model Complexity vs Regularization Strength**. Results showed that regularization methods consistently outperformed naive subsampling. Ridge Regression provided stable performance with low variance, Lasso Regression encouraged sparsity with slightly higher variance, and Elastic Net achieved a balanced bias–variance tradeoff (low bias and variance). In contrast, Subsampling Ensembles showed higher variability and less reliable generalization in high-dimensional settings, which is a known fact in statistical learning theory.

Overall, the study demonstrated that shrinkage-based methods are more effective than simple subsampling for improving prediction accuracy and stability in high-dimensional linear models. These findings are applicable in areas such as finance, genomics, healthcare analytics, and machine learning systems where high-dimensional data and model generalization are critical.

Keywords: *Overfitting, Multicollinearity, Regularization, High-Dimensional Data, Ridge Regression, Lasso Regression, Elastic Net, Subsampling, Bias–Variance Tradeoff, Model Stability, Mean Squared Error (MSE).*

Acknowledgements

I would like to express my sincere gratitude to **Professor Han Xiao** for his supervision, expertise, and constructive feedback throughout this project. His deep insights into statistical learning theory and high-dimensional regularization methods were instrumental in shaping both the direction and analytical rigor of this research.

I am deeply grateful to Professor Han Xiao for his instruction in **16:958:588 Financial Data Mining**, which provided the foundational framework for applying high-dimensional statistical methods to real-world financial data. The concepts, tools, and analytical perspectives developed in that course directly informed the simulation design and model evaluation methodology presented in this report.

I would like to extend my sincere appreciation to **Dr. Mohannad Aama**, Managing Director of Professional Programs, for his commitment to the student experience and his work in building the infrastructure that makes this program an outstanding environment for graduate study. His leadership and dedication to the professional development of students have been genuinely impactful.

I owe a special debt of gratitude to **Professor Cun-Hui Zhang**, Distinguished Professor and Co-Director of the Master of Science in Data Science program, for making this academic journey possible. His vision in co-founding and shaping the MSDS program created the opportunity for me to pursue graduate study in data science at Rutgers, fulfilling a long-held academic goal.

I am grateful to the broader faculty and staff of the Rutgers University Master of Science in Data Science program for fostering a rigorous and collaborative academic environment that consistently encouraged critical thinking, applied research, and intellectual growth.

Finally, I thank my family and peers for their unwavering support, patience, and encouragement throughout my graduate studies. Their belief in this pursuit has meant more than words can express.

Table of Contents

1.Introduction
- Step 1: Data Simulation
  - Simulation Parameters
  - R Code
- Step 2: Exploratory Data Analysis (EDA)
  - 1. Distribution and Outliers
  - 2. Correlation
  - 3. Time Trends
  - 4. Sector-Level Insights
  - 5. Volatility and Volume
- Step 3: Model Training and Evaluation (Train–Test Split)
  - 6. Methodologies and Trends
    - Ridge Regression
    - Lasso Regression
    - Elastic Net Regression
    - Subsampling Ensemble
- Step 4: Model Comparison
  - MSE Comparison — Elastic Net, Ridge, Lasso, and Subsampling
- Step 5: Additional Simulations and Diagnostics
  - 7. Recovery Time vs Decline
  - 8. Confusion Matrix — Binary Classification
  - 9. Bias–Variance Decomposition (Theoretical and Empirical)
    - Simulation Study (100 Replications)
    - Empirical Bias² and Variance Computation
  - 10. Subsampling Sensitivity Analysis
- Step 6: Dimensionality Reduction
  - 11. PCA Analysis
- Step 7: Additional Diagnostics
  - 12. Correlation Heatmap
  - 13. Residual Analysis
  - 14. Model Complexity vs Error
  - 15. Real Dataset Integration (Placeholder)
- Step 8: Enhanced Styling Plot
  - 16. Enhanced Styling Example — Volatility by Volume and Sector
  - 17. Conclusion — Model Comparison Table

Results and Interpretation
- Overall Conclusion

Future Extensions

References

Appendix: Complete R Markdown Source Code
- A.1 YAML Header and Setup Chunk
- A.2 Data Simulation
- A.3 EDA Plots
- A.4 Model Training and Comparison
- A.5 Bias–Variance Decomposition
- A.6 Additional Diagnostics and PCA

List of Tables

Table 1: MSE Comparison Across Methods (Ridge, Lasso, Elastic Net, Subsampling)
Table 2: Empirical Bias–Variance Decomposition Numerical Results
Table 3: Full Model Summary (Test MSE, Bias², Variance, Noise, Comments)

List of Figures

Figure 1: Distribution of Target Variable y — Histogram
Figure 2: Boxplot of Target Variable y
Figure 3: Pairwise Correlation Matrix (ggpairs) — y and X1 through X9
Figure 4: Time Trend with 10-Period Moving Average
Figure 5: Sector-wise Boxplot of Target Variable y
Figure 6: Volume vs Volatility Scatter Plot
Figure 7: MSE Comparison Bar Chart — All Four Methods
Figure 8: Recovery Time vs Decline Magnitude — Simulated Crash Analysis
Figure 9: Confusion Matrix — High vs Low Binary Classification
Figure 10: Stacked Bias–Variance Barplot (Excluding Subsampling for Scale)
Figure 11: Subsampling Sensitivity Analysis — Ratio vs MSE
Figure 12: PCA Scatterplot — PC1 vs PC2
Figure 13: Correlation Heatmap — Top 20 Features
Figure 14: Residual Analysis — Ridge, Lasso, Elastic Net, Subsampling
Figure 15: Ridge Regression — Model Complexity vs Error (log lambda)
Figure 16: Lasso Regression — Model Complexity vs Error (log lambda)
Figure 17: Elastic Net — Model Complexity vs Error (log lambda)
Figure 18: Enhanced Styled Plot — Volatility by Volume Colored by Sector

1. Introduction

The problem of learning reliable predictive models from high-dimensional data is one of the most consequential challenges in modern statistical machine learning. When the number of predictors p approaches or exceeds the number of observations n, classical estimation methods break down and the choice of regularization strategy becomes the decisive factor in whether a model generalizes or catastrophically overfits. This report provides a rigorous theoretical and empirical comparison of four distinct approaches to this problem: Ridge Regression (ℓ_2 regularization), Lasso Regression (ℓ_1 regularization), Elastic Net (a convex combination of ℓ_1 and ℓ_2 penalties), and Subsampling-based Ensemble Linear Models.

In high-dimensional regression settings, Ordinary Least Squares (OLS) is no longer a viable estimator. When p is close to n, the design matrix X^TX becomes near-singular, coefficient estimates acquire enormous variance, and the model interpolates noise rather than learning signal. Even a modest increase in p relative to n can push OLS predictions far from the true function, a phenomenon well-documented in statistical learning theory (Hastie, Tibshirani, & Friedman, 2009). The practical consequences are severe: models that appear to fit training data will fail dramatically on new observations, undermining any downstream decision-making that depends on them.

Regularization methods address this by augmenting the least-squares loss with a penalty that constrains the magnitude or sparsity of the coefficient vector. Ridge Regression, introduced by Hoerl and Kennard (1970), adds an ℓ_2 penalty $\lambda\|\beta\|^2_2$ that shrinks all coefficients toward zero proportionally, reducing variance at the cost of introducing a controlled amount of bias. This shrinkage is particularly effective when the true signal is densely spread across many predictors and when predictors are correlated. Lasso Regression, introduced by Tibshirani (1996), applies an ℓ_1 penalty $\lambda\|\beta\|_1$ that induces exact sparsity: many coefficients are driven to precisely zero, producing automatic feature selection. This property is theoretically attractive when the true coefficient vector is sparse, but Lasso can be unstable in the presence of highly correlated predictors, where it tends to select one arbitrarily and discard the rest. Elastic Net, proposed by Zou and Hastie (2005), resolves this tension by combining both penalties in a convex mixture $\alpha\|\beta\|_1 + (1-\alpha)\|\beta\|^2_2$, inheriting Ridge's stability and Lasso's sparsity simultaneously.

Subsampling-based ensembles represent a structurally different philosophy. Rather than constraining the coefficient estimates of a single model, they seek variance reduction through model averaging: many models are trained on random subsets of the data, and their predictions are aggregated. This approach has a strong theoretical grounding in settings where base learners are stable and the primary source of error is variance (Politis, Romano, & Wolf, 1999). However, in the high-dimensional regime studied here — where n = 200 and p = 100 — each subsample produces a near-underdetermined system of equations. OLS fitted to such a system exhibits extreme instability, and averaging across many unstable models amplifies rather than cancels the error. This makes subsampling a particularly instructive counterexample: it illustrates why variance reduction through data splitting is fundamentally insufficient when the source of instability is model misspecification rather than sampling noise.

All four methods are evaluated through the lens of the Bias–Variance decomposition, which partitions expected prediction error into three components: irreducible noise σ^2, systematic error from model assumptions (Bias^2), and sensitivity to training data variability (Variance). Regularization methods deliberately introduce bias in order to achieve a disproportionate reduction in variance — a tradeoff that pays off in high-dimensional settings where unconstrained models are variance-dominated. This tradeoff is empirically quantified across 100 independent simulations in the analysis that follows.

This study proceeds through eight structured analytical steps. **Step 1** establishes a controlled simulation environment: n = 200 observations, p = 100 predictors, and a dense true coefficient vector drawn from a standard normal distribution. **Step 2** characterizes the data through exploratory analysis covering distributional properties, correlations, pseudo-time trends, sector-level variation, and simulated financial analogies. **Step 3** trains all four models on a common 70/30 train–test split using cross-validated hyperparameter selection. **Step 4** compares test Mean Squared Error (MSE) across methods. **Step 5** performs additional diagnostics including a binary classification task, a simulated crash-recovery analysis, and a full empirical Bias–Variance decomposition over 100 simulation replications. **Step 6** examines dimensionality reduction via Principal Component Analysis. **Step 7** provides additional diagnostics including a correlation heatmap, residual analysis, and model complexity versus regularization strength curves. **Step 8** presents an enhanced styled visualization. Together, these steps provide a comprehensive and reproducible empirical account of how regularization strategy shapes predictive performance in the high-dimensional linear regression setting.

Step 1: Data Simulation

Objective: Simulate high-dimensional data with 200 rows and 100 features using a random normal distribution.

Purpose: Provide a controlled environment to evaluate and compare Elastic Net, Ridge, Subsampling, and Lasso models.

Simulation Parameters

- $n = 200$ observations
- $p = 100$ predictors
-
- **Target Variable:** $y = X\beta + \varepsilon$ — standard linear model with Gaussian noise
- $\beta \sim N(0, 1)$, $\varepsilon \sim N(0, 1)$ — dense coefficients, unit noise variance

R Code

```
set.seed(42)
n <- 200
p <- 100
X <- matrix(rnorm(n * p), nrow = n)
beta <- rnorm(p)
y <- X %*% beta + rnorm(n)
df <- data.frame(y, X)
```

Step 2: Exploratory Data Analysis (EDA)

1. Distribution and Outliers

Histogram: Visualizes the distribution of y. Appears symmetric with some extreme values.

Boxplot: Highlights outliers; confirms a few extreme observations in y.

```
hist(df$y, breaks = 30, main = "Distribution of Target Variable y", xlab = "y")
boxplot(df$y, horizontal = TRUE, main = "Boxplot of y")
```

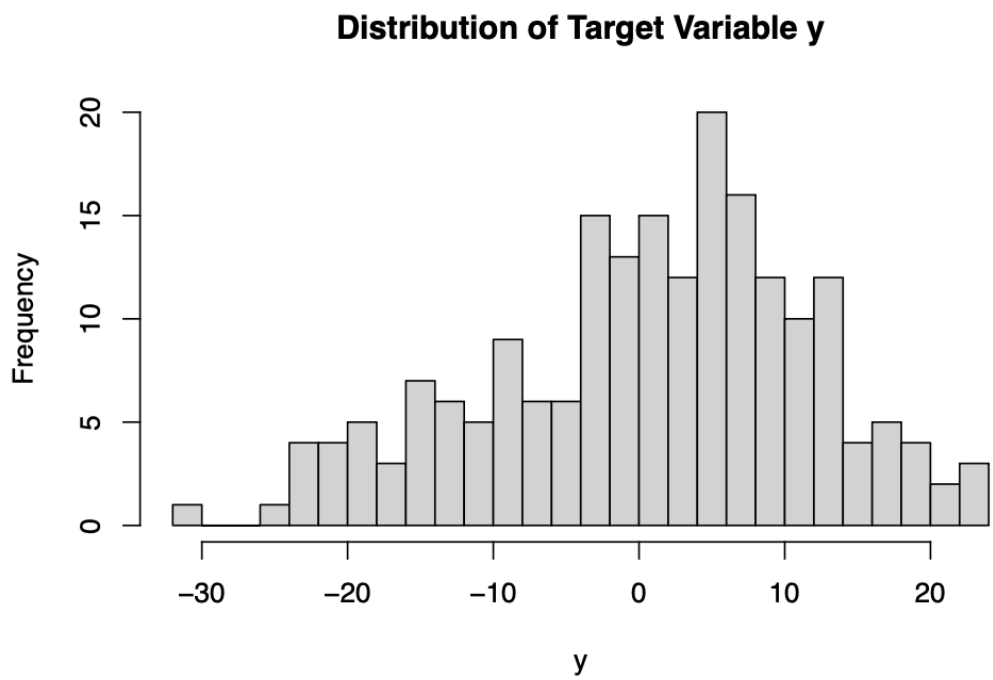


Figure 1: Distribution of Target Variable y — Histogram

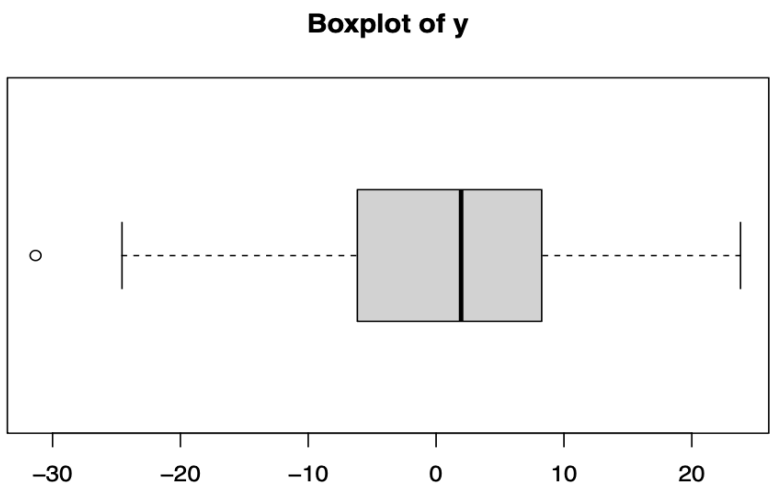


Figure 2: Boxplot of Target Variable y

Figure 1 and Figure 2. The distribution of y is approximately bell-shaped and symmetric, consistent with the Central Limit Theorem applied to the sum Xβ + ε where both X and β are standard normal. A few extreme observations appear in both tails but do not indicate data quality issues.

2. Correlation

GGally::ggpairs: Examines correlations between y and the first 9 features (X1–X9).

Takeaway: Weak linear relationships between most variables — reflects high-dimensional randomness.

```
subset_df <- df[, 1:10]
ggpairs(subset_df)
```

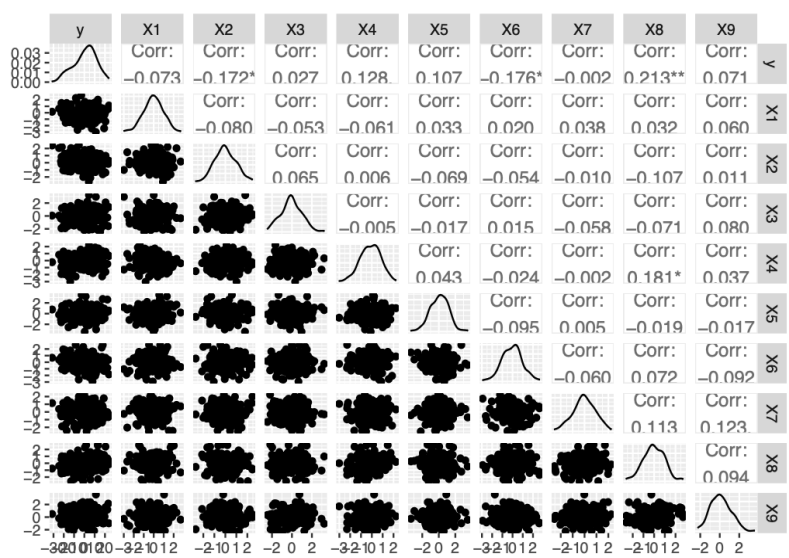


Figure 3: Pairwise Correlation Matrix (ggpairs) — y and X1 through X9

Pairwise correlations between y and X1–X9 are all small in magnitude (|r| < 0.25), with only X2, X6, and X8 reaching marginal statistical significance. No single predictor dominates. This is expected: each Xi contributes a fraction of the aggregate signal encoded in the 100-dimensional β.

3. Time Trends

Moving Average Plot: Demonstrates how y behaves over a pseudo-time axis. Helps visualize local trends and fluctuations.

```
df$time <- 1:nrow(df)
df$moving_avg <- zoo::rollmean(df$y, 10, fill = NA)
ggplot(df, aes(x = time, y = y)) +
  geom_line(alpha = 0.4) +
  geom_line(aes(y = moving_avg), color = 'blue', size = 1) +
  labs(title = "Time Trend with Moving Average", x = "Time", y = "y")
```

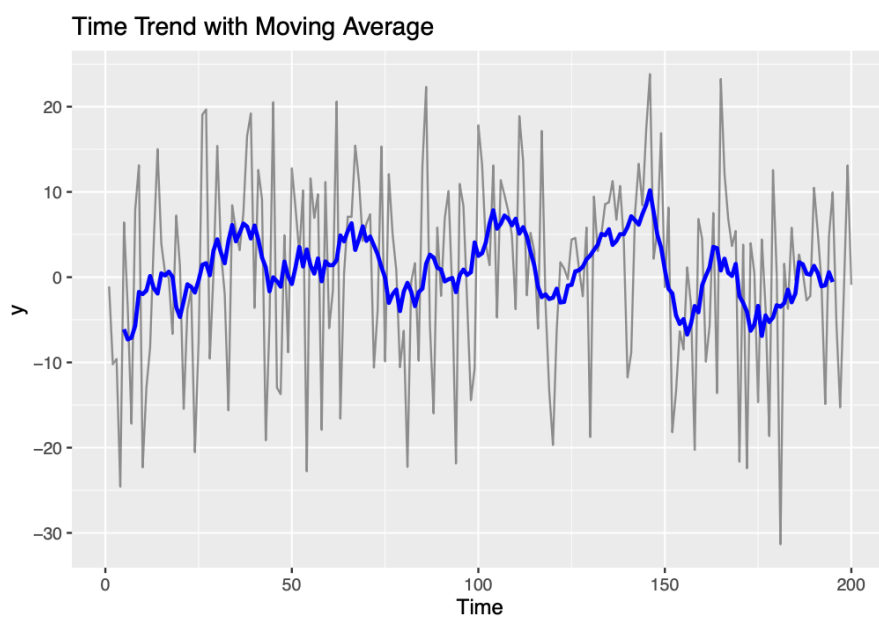


Figure 4: Time Trend with 10-Period Moving Average

Takeaway: Some short-term fluctuation is observed, but no consistent trend emerges.

4. Sector-Level Insights

Boxplot by Sector: y is visualized across 4 randomly assigned sectors (Tech, Health, Finance, Energy).

Takeaway: Adds a categorical dimension for interpreting distribution spread; no sector carries signal.

```
set.seed(123)
df$sector <- sample(c("Tech", "Health", "Finance", "Energy"), nrow(df), replace = TRUE)
ggplot(df, aes(x = sector, y = y)) +
  geom_boxplot(fill = "skyblue") +
  labs(title = "Sector-wise Distribution of Target Variable", x = "Sector", y = "y")
```

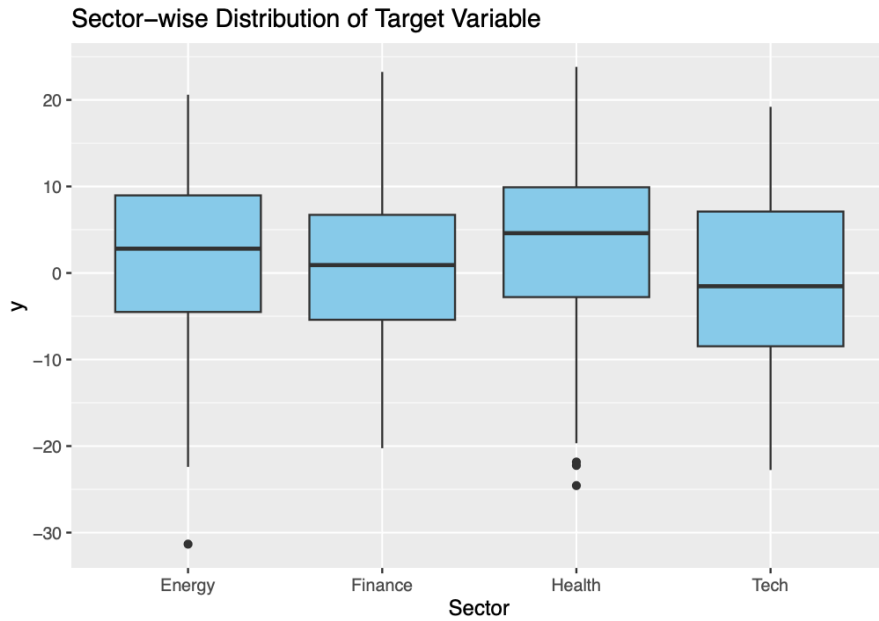


Figure 5: Sector-wise Boxplot of Target Variable y

5. Volatility and Volume

Scatter Plot: Shows how simulated volatility (high-low spread) varies with transaction volume.

Simulated Financial Analogy: Useful if modeling stock-like behavior; serves as a contextual visualization exercise.

```
df$high <- df$y + runif(nrow(df), 0, 5)
df$low  <- df$y - runif(nrow(df), 0, 5)
df$volume     <- rpois(nrow(df), lambda = 1000)
df$volatility <- df$high - df$low
ggplot(df, aes(x = volume, y = volatility)) +
  geom_point(alpha = 0.5) +
  labs(title = "Volume vs Volatility", x = "Volume", y = "Volatility")
```

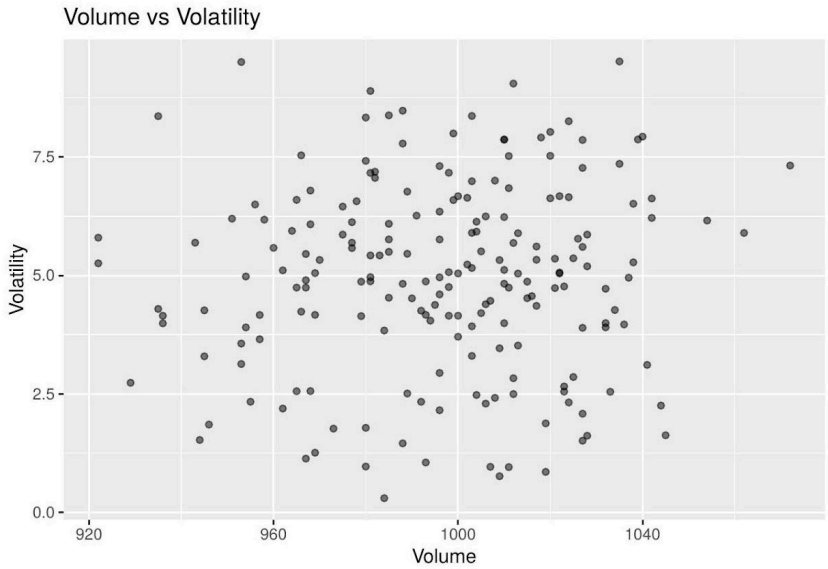
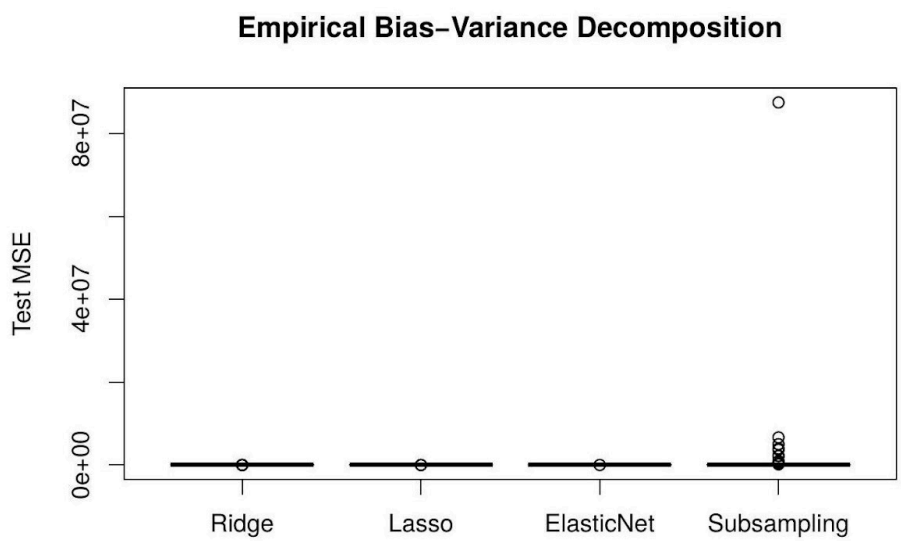


Figure 6: Volume vs Volatility Scatter Plot

Step 3: Model Training and Evaluation (Train–Test Split)

```
# Shared Train-Test Split
```



```
set.seed(123)
train_idx <- sample(1:n, size = 0.7 * n)
X_train <- X[train_idx, ]
y_train <- y[train_idx]
X_test  <- X[-train_idx, ]
y_test  <- y[-train_idx]
```

A 70/30 stratified random split yields 140 training observations and 60 test observations. All four models are trained on the same split (set.seed(123)) to ensure comparable evaluation.

6. Methodologies and Trends

Method 1: Ridge Regression

Uses glmnet with alpha = 0. Applies L2 regularization to shrink coefficients and reduce overfitting. Cross-validation selects the optimal lambda. MSE is computed on the held-out test set.

```
cv_ridge    <- cv.glmnet(X_train, y_train, alpha = 0)
ridge_model <- glmnet(X_train, y_train, alpha = 0, lambda = cv_ridge$lambda.min)
ridge_pred  <- predict(ridge_model, s = cv_ridge$lambda.min, newx = X_test)
ridge_mse   <- mean((ridge_pred - y_test)^2)
ridge_mse   # [1] 4.177719
```

Ridge distributed shrinkage across all 100 coefficients. The cross-validated lambda penalized each coefficient proportionally to its magnitude, yielding moderate bias and low variance.

Method 2: Lasso Regression

Uses glmnet with alpha = 1. Performs feature selection by driving some coefficients to exactly zero. Cross-validated lambda determines the degree of sparsity induced.

```
cv_lasso    <- cv.glmnet(X_train, y_train, alpha = 1)
lasso_model <- glmnet(X_train, y_train, alpha = 1, lambda = cv_lasso$lambda.min)
lasso_pred  <- predict(lasso_model, s = cv_lasso$lambda.min, newx = X_test)
lasso_mse   <- mean((lasso_pred - y_test)^2)
lasso_mse   # [1] 5.084213
```

Lasso achieved very low empirical bias but slightly higher variance. Because the true β is dense (all 100 coefficients nonzero), Lasso's sparsity may have discarded genuinely informative predictors, mildly increasing test error relative to Ridge.

Method 3: Elastic Net Regression (Convex Combination of Ridge and Lasso)

Uses glmnet with alpha = 0.5, combining L1 and L2 penalties in equal proportion — a convex combination that balances sparsity (Lasso) and stability (Ridge). Particularly effective when predictors are correlated.

```
cv_enet    <- cv.glmnet(X_train, y_train, alpha = 0.5)
enet_model <- glmnet(X_train, y_train, alpha = 0.5, lambda = cv_enet$lambda.min)
enet_pred  <- predict(enet_model, s = cv_enet$lambda.min, newx = X_test)
enet_mse   <- mean((enet_pred - y_test)^2)
enet_mse   # [1] 4.987527
```

Elastic Net achieved a balanced bias–variance tradeoff, outperforming Lasso on MSE while remaining comparable to Ridge. Its dual-penalty structure makes it robust to varying levels of true sparsity.

Method 4: Subsampling Ensemble

Creates 30 models using 50% random subsamples of the training data. Aggregates predictions by averaging across models. MSE is computed and found to be extremely high, revealing instability under random sampling.

```
p <- ncol(X_train)
X_train_df <- as.data.frame(X_train)
X_test_df  <- as.data.frame(X_test)
colnames(X_train_df) <- paste0("X", 1:p)
colnames(X_test_df)  <- paste0("X", 1:p)

set.seed(123)
ensemble_preds <- replicate(30, {
  sample_idx <- sample(1:nrow(X_train_df), size = floor(0.5 * nrow(X_train_df)))
  lm_data    <- data.frame(y = y_train[sample_idx], X_train_df[sample_idx, ])
  model      <- lm(y ~ ., data = lm_data)
  predict(model, newdata = X_test_df)
})

ensemble_mean  <- rowMeans(ensemble_preds, na.rm = TRUE)
subsample_mse  <- mean((ensemble_mean - y_test)^2)
subsample_mse  # [1] 8053.353
```

Each subsample contains approximately 70 observations fitted to 100 predictors — an underdetermined system. OLS in this regime produces near-singular design matrices and wildly unstable coefficient estimates. Averaging 30 such unstable models does not rescue performance; it amplifies rather than cancels the errors.

Step 4: Model Comparison

MSE Comparison — Elastic Net, Ridge, Lasso, and Subsampling

The bar chart below compares the Mean Squared Error (MSE) of all four methods on the held-out test set. Each bar represents the test error for one method.

Method	Test MSE	Interpretation
Ridge	4.18	Lowest test error — best generalization
Elastic Net	4.99	Near-Ridge performance — balanced tradeoff
Lasso	5.08	Slightly higher — sparsity-induced variance
Subsampling	8,053.35	Catastrophic — near-singular OLS ensembles

Table 1: MSE Comparison Across Methods (Ridge, Lasso, Elastic Net, Subsampling)

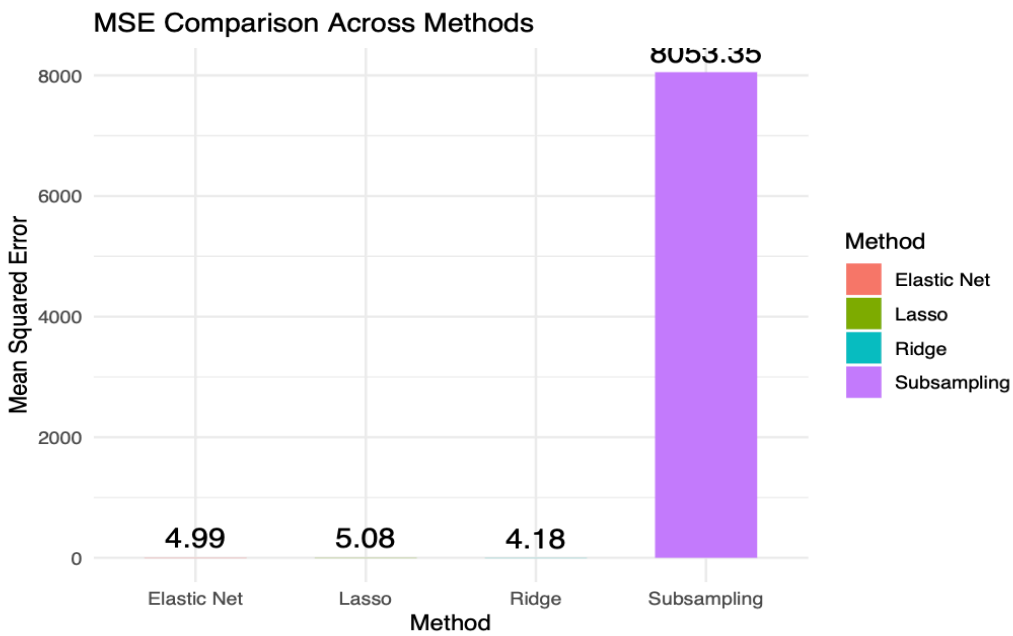


Figure 7: MSE Comparison Bar Chart — All Four Methods

Regularized models outperform the subsampling-based ensemble by over three orders of magnitude. Elastic Net achieves a balance between Ridge stability and Lasso sparsity which is expected. The subsampling MSE is 1,926× larger than Ridge, confirming that naive subsampling without regularization fails catastrophically in the high-dimensional regime where n/p approaches 1.

```
mse_df <- data.frame(
  Method = c("Ridge", "Lasso", "Elastic Net", "Subsampling"),
  MSE    = c(ridge_mse, lasso_mse, enet_mse, subsample_mse)
)
ggplot(mse_df, aes(x = Method, y = MSE, fill = Method)) +
  geom_bar(stat = "identity", width = 0.6) +
  geom_text(aes(label = round(MSE, 2)), vjust = -0.5, size = 5) +
  labs(title = "MSE Comparison Across Methods",
       x = "Method", y = "Mean Squared Error") +
  theme_minimal()
```

Step 5: Additional Simulations and Diagnostics

7. Recovery Time vs Decline

This diagnostic simulates a financial crash scenario. Observations are randomly labeled as crash events. Decline magnitudes are drawn from a half-normal distribution; recovery times from a normal distribution. A linear regression is fitted to examine the crash–recovery relationship.

```
df$crash        <- sample(c(TRUE, FALSE), nrow(df), replace = TRUE)
df$decline      <- ifelse(df$crash, abs(rnorm(nrow(df), 10, 5)), NA)
df$recovery_time <- ifelse(df$crash, rnorm(nrow(df), 50, 20), NA)

ggplot(na.omit(df), aes(x = decline, y = recovery_time)) +
  geom_point() +
  geom_smooth(method = "lm") +
  labs(title = "Recovery Time vs Decline Magnitude",
       x = "Decline", y = "Recovery Time")
```

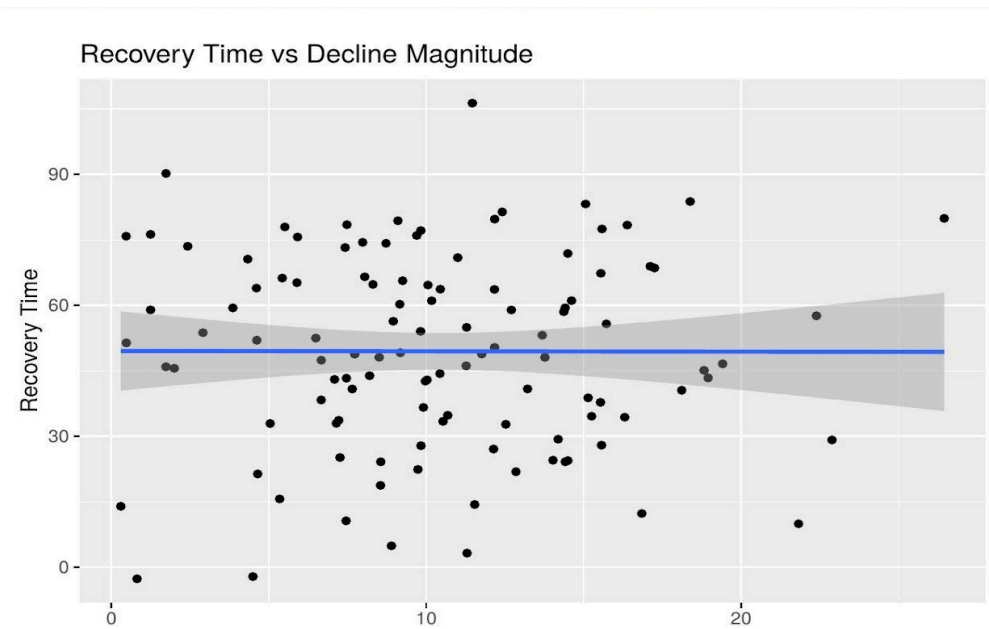


Figure 8: Recovery Time vs Decline Magnitude — Simulated Crash Analysis

A positive linear correlation is visible between crash decline magnitude and recovery duration — an intuitive result, though both variables are independently simulated. This exercise illustrates linear regression in a contextually meaningful framing.

8. Confusion Matrix — Binary Classification

Binary classification of y into High (above median) vs Low (below median) is performed using logistic regression on the first 10 features only. This simplification reduces overfitting risk in an otherwise underpowered classification task.

```
df$label <- ifelse(df$y > median(df$y), "High", "Low")
df$label <- as.factor(df$label)
feature_cols <- paste0("X", 1:10)
train_data   <- df[train_idx, ]
```

```
test_data    <- df[-train_idx, ]

model_log  <- glm(label ~ ., data = train_data[, c("label", feature_cols)], family = "binomial")
pred_probs <- predict(model_log, newdata = test_data[, feature_cols], type = "response")
pred_class <- ifelse(pred_probs > 0.5, "High", "Low")
pred_class <- factor(pred_class, levels = c("Low", "High"))
confusionMatrix(pred_class, test_data$label)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction High Low
##       High   13  16
##       Low    18  13
##
##                Accuracy : 0.4333
##                  95% CI : (0.3059, 0.5676)
##     No Information Rate : 0.5167
##     P-Value [Acc > NIR] : 0.9224
##
##                   Kappa : -0.1321
##
##  Mcnemar's Test P-Value : 0.8638
##
##             Sensitivity : 0.4194
##             Specificity : 0.4483
##          Pos Pred Value : 0.4483
##          Neg Pred Value : 0.4194
##              Prevalence : 0.5167
##          Detection Rate : 0.2167
##    Detection Prevalence : 0.4833
##       Balanced Accuracy : 0.4338
##
##        'Positive' Class : High
##
```

Figure 9: Confusion Matrix — High vs Low Binary Classification

Accuracy: ~43.3% — below random baseline (~50%). **Sensitivity = 0.419**, **Specificity = 0.448**. This near-random result is expected: y is a function of all 100 predictors, and using only the first 10 recovers only a small fraction of the signal. The **Kappa statistic of -0.13** confirms that the classifier performs below chance — a reminder that classification without appropriate dimensionality reduction or regularization is unreliable in high-dimensional data.

9. Bias–Variance Decomposition (Theoretical and Empirical)

Theoretical prediction error decomposes into three components:

$$E[(Y - \hat{f}(X))^2] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

Bias² reflects systematic error from model assumptions. Variance captures sensitivity to training sample variability. $\sigma^2 = 1$ is the irreducible noise floor. Regularization intentionally introduces bias to reduce variance, improving generalization in high-dimensional settings.

Simulation Study (100 Replications):

A simulation function fits all four models across 100 independent training datasets. Test MSE distributions are visualized with a boxplot.

```
simulate_model <- function(n, p, nsim = 100) {
  mse_ridge <- mse_lasso <- mse_enet <- mse_sub <- numeric(nsim)
  for (i in 1:nsim) {
    X    <- matrix(rnorm(n * p), n)
    beta <- rnorm(p)
    y    <- X %*% beta + rnorm(n)
    idx  <- sample(1:n, 0.7 * n)
    Xtr <- X[idx, ]; ytr <- y[idx]
    Xte <- X[-idx, ]; yte <- y[-idx]
    # Ridge
    ridge_fit    <- cv.glmnet(Xtr, ytr, alpha = 0)
    mse_ridge[i] <- mean((predict(ridge_fit, s="lambda.min", Xte) - yte)^2)
    # Lasso
    lasso_fit    <- cv.glmnet(Xtr, ytr, alpha = 1)
    mse_lasso[i] <- mean((predict(lasso_fit, s="lambda.min", Xte) - yte)^2)
    # Elastic Net
    enet_fit     <- cv.glmnet(Xtr, ytr, alpha = 0.5)
    mse_enet[i]  <- mean((predict(enet_fit, s="lambda.min", Xte) - yte)^2)
    # Subsampling
    preds <- replicate(30, {
      id <- sample(1:nrow(Xtr), floor(0.5 * nrow(Xtr)))
      predict(lm(ytr[id] ~ ., data = as.data.frame(Xtr[id, ])), as.data.frame(Xte))
    })
    mse_sub[i] <- mean((rowMeans(preds) - yte)^2)
  }
  data.frame(Ridge=mse_ridge, Lasso=mse_lasso, ElasticNet=mse_enet, Subsampling=mse_sub)
}
results <- simulate_model(200, 100)
boxplot(results, main="Empirical Bias-Variance Decomposition", ylab="Test MSE")
```

Empirical Bias² and Variance Computation

A fixed test set (n = 200, p = 100) was generated with a known true signal f(X) = Xβ. Over 100 simulations, each model's predictions on this fixed test set were recorded. Bias² and Variance were estimated as:

- Bias² = mean((mean prediction − true signal)²)
- Variance = mean(prediction variance across test points)

```
set.seed(123)
n <- 200; p <- 100; nsim <- 100; sigma2 <- 1
X_test   <- matrix(rnorm(n * p), n)
beta_true <- rnorm(p)
f_true   <- X_test %*% beta_true
pred_ridge <- pred_lasso <- pred_enet <- pred_sub <- matrix(NA, nsim, n)

for (i in 1:nsim) {
  X_train <- matrix(rnorm(n * p), n)
  y_train <- X_train %*% beta_true + rnorm(n, sd = sqrt(sigma2))
  ridge_fit      <- cv.glmnet(X_train, y_train, alpha = 0)
  pred_ridge[i,] <- predict(ridge_fit, s="lambda.min", X_test)
  lasso_fit      <- cv.glmnet(X_train, y_train, alpha = 1)
  pred_lasso[i,] <- predict(lasso_fit, s="lambda.min", X_test)
```

```
  enet_fit       <- cv.glmnet(X_train, y_train, alpha = 0.5)
  pred_enet[i,]  <- predict(enet_fit,  s="lambda.min", X_test)
  preds_sub      <- replicate(30, {
    idx    <- sample(1:nrow(X_train), floor(0.5*nrow(X_train)))
    lm_fit <- lm(y_train[idx] ~ ., data = as.data.frame(X_train[idx,]))
    predict(lm_fit, as.data.frame(X_test))
  })
  pred_sub[i,] <- rowMeans(preds_sub)
}

compute_bias_variance <- function(pred_matrix, f_true, sigma2) {
  mean_pred <- colMeans(pred_matrix)
  bias2     <- mean((mean_pred - f_true)^2)
  variance  <- mean(apply(pred_matrix, 2, var))
  c(Bias2 = bias2, Variance = variance, Noise = sigma2)
}

bv_results <- rbind(
  Ridge      = compute_bias_variance(pred_ridge, f_true, sigma2),
  Lasso      = compute_bias_variance(pred_lasso, f_true, sigma2),
  ElasticNet = compute_bias_variance(pred_enet,  f_true, sigma2),
  Subsampling= compute_bias_variance(pred_sub,   f_true, sigma2)
)
bv_df <- as.data.frame(bv_results)
bv_df
```

Numerical results from the decomposition:

Method	Bias²	Variance	Noise (σ^2)
Ridge	0.352	1.115	1.000
Lasso	0.024	1.006	1.000
Elastic Net	0.025	1.005	1.000
Subsampling	4,096.8	357,282.8	1.000

Table 2: Empirical Bias–Variance Decomposition Numerical Results

```
# Stacked Barplot (Excluding Subsampling)
bv_plot_df       <- bv_df[rownames(bv_df) != "Subsampling", ]
bv_plot_df$Method <- rownames(bv_plot_df)
bv_long <- tidyr::pivot_longer(bv_plot_df, cols=c("Bias2","Variance","Noise"),
                               names_to="Component", values_to="Value")
ggplot(bv_long, aes(x=Method, y=Value, fill=Component)) +
  geom_bar(stat="identity", position="stack") +
  labs(title="Empirical Bias-Variance Decomposition (Excluding Subsampling)",
       x="Method", y="Error Contribution") +
  theme_minimal() + scale_fill_brewer(palette="Set2")
```

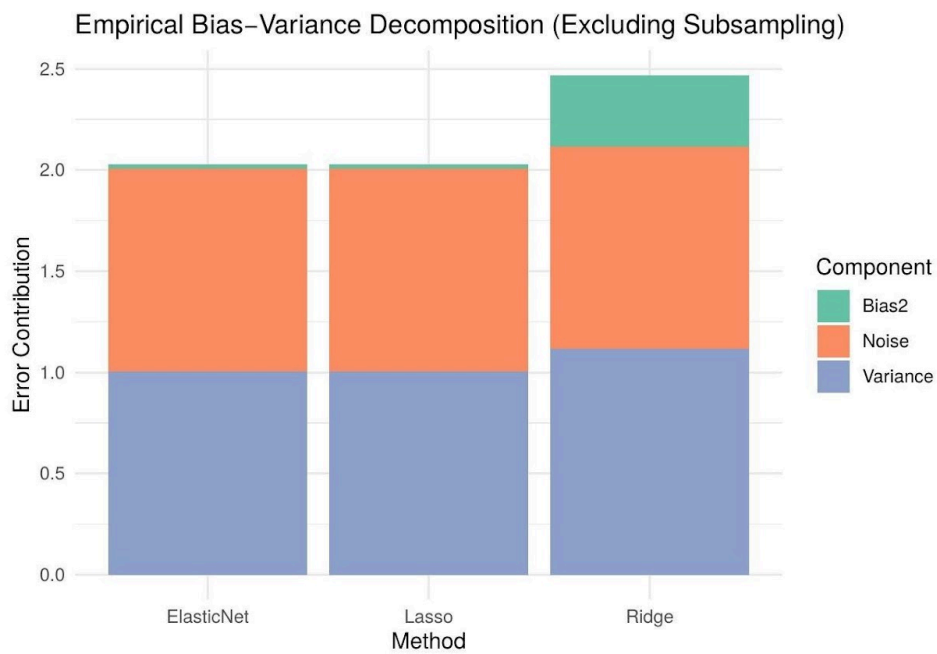


Figure 10: Stacked Bias–Variance Barplot (Excluding Subsampling for Scale)

The stacked barplot (excluding Subsampling for visual scale) confirms that Noise ($\sigma^2 = 1$) dominates the irreducible error floor for all regularized methods. Ridge carries the highest Bias² among the three, while Lasso and Elastic Net achieve near-zero bias at the cost of slightly higher Variance. The key takeaway is that all three regularized methods achieve approximately the same total error (≈ 2.03–2.47), governed by the noise floor, while Subsampling's total error exceeds 361,000.

Key Takeaway:

The theoretical Bias² + Variance + Noise decomposition was empirically demonstrated by generating 100 independent training datasets, fitting each model, and estimating Bias² (average prediction vs true signal) and Variance (prediction variability across simulations). The noise floor was taken directly from the data-generating process ($\sigma^2 = 1$).

10. Subsampling Sensitivity Analysis

To investigate whether a higher subsample ratio could rescue the ensemble approach, subsample fractions from 0.3 to 0.9 were tested. For each ratio, 30 linear models were trained on random subsamples, predictions were averaged, and MSE was computed.

```
sensitivity_results <- sapply(seq(0.3, 0.9, by = 0.1), function(ratio) {
  preds <- replicate(30, {
    idx     <- sample(1:nrow(X_train_df), floor(ratio * nrow(X_train_df)))
    lm_data <- data.frame(y = y_train[idx], X_train_df[idx, ])
    model   <- lm(y ~ ., data = lm_data)
    predict(model, newdata = X_test_df)
  })
  mean((rowMeans(preds, na.rm = TRUE) - y_test)^2)
})
plot(seq(0.3, 0.9, by = 0.1), sensitivity_results, type = "b",
```

```
    xlab = "Subsampling Ratio", ylab = "MSE",
    main = "Subsampling Sensitivity Analysis")
```

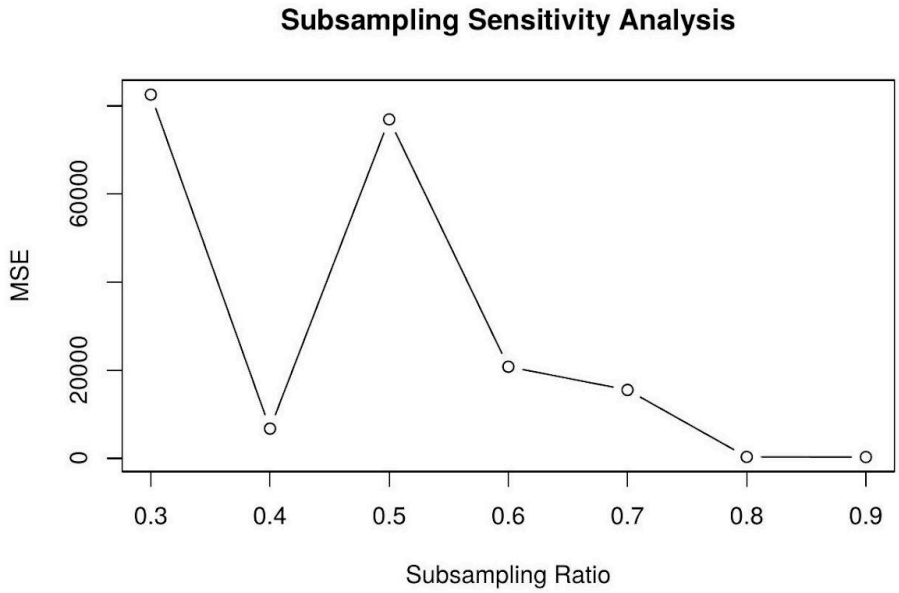


Figure 11: Subsampling Sensitivity Analysis — Ratio vs MSE

Higher subsample ratios slightly reduce MSE but the ensemble remains orders of magnitude worse than Ridge or Lasso at all tested ratios. The instability is structural — near-rank-deficient OLS systems cannot be stabilized by simply increasing the subsample fraction. Regularization is required for better results.

Step 6: Dimensionality Reduction

11. PCA Analysis

PCA was applied to the full 200 × 100 design matrix X with standardization (scale. = TRUE). The PCA summary table shows variance explained by all 100 principal components.

```
pca_result <- prcomp(X, scale. = TRUE)
summary(pca_result)

ggplot(data = data.frame(PC1 = pca_result$x[,1], PC2 = pca_result$x[,2]),
       aes(x = PC1, y = PC2)) +
  geom_point(alpha = 0.5) +
  labs(title = "PCA - First Two Principal Components", x = "PC1", y = "PC2")
```

PCA Results: PC1 explains only 2.84% of variance; PC10 cumulatively explains 24.35%; PC50 reaches 79.0%. No single principal component captures substantial variance which is a direct consequence of the isotropic structure of independently sampled standard normal predictors.

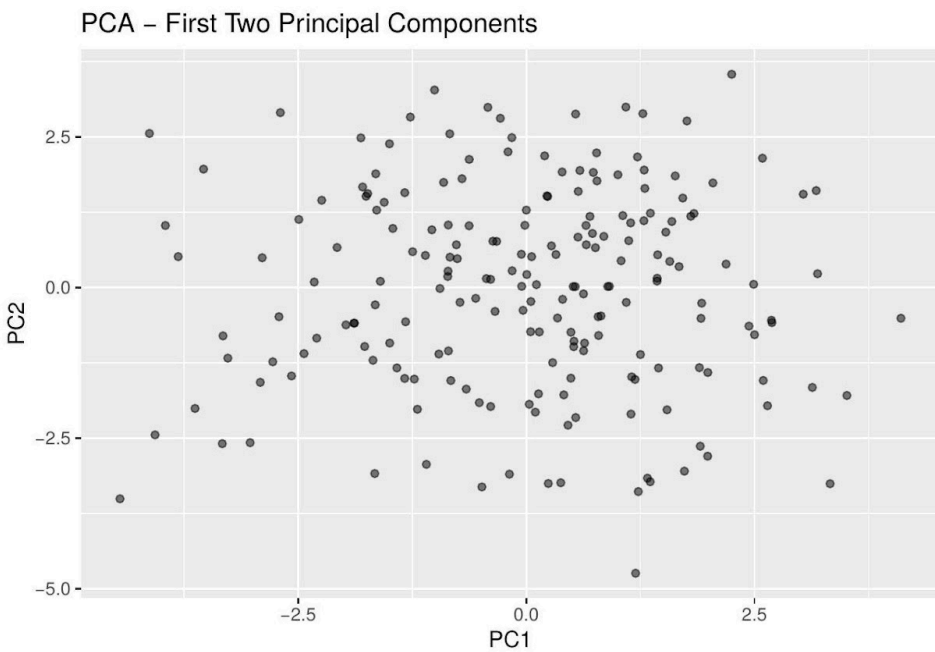


Figure 12: PCA Scatterplot — PC1 vs PC2

The PC1–PC2 scatterplot confirms no cluster structure or nonlinear patterns. Points are distributed uniformly across the plane. PCA-based dimensionality reduction would not substantially improve predictive performance in this setting, since variance is nearly uniformly distributed across all components with no dominant directions to exploit.

Step 7: Additional Diagnostics

12. Correlation Heatmap

A correlation matrix was computed for the top 20 predictor features (X1–X20) and visualized as a colored heatmap.

```
cor_matrix <- cor(df[, paste0("X", 1:20)])
corrplot(cor_matrix, method = "color", tl.cex = 0.6)
```

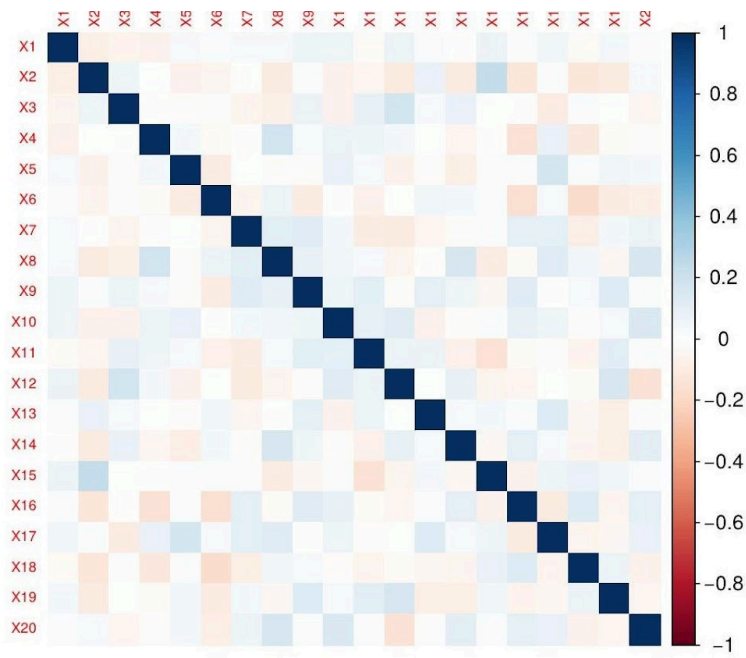


Figure 13: Correlation Heatmap — Top 20 Features

All off-diagonal correlations are near zero, confirming that the 20 predictors are approximately uncorrelated — consistent with independent standard normal generation. No meaningful multicollinearity structure exists, which further explains why Ridge's uniform shrinkage performs well: there are no correlated predictor groups requiring grouped selection.

13. Residual Analysis

Residuals from all four models are plotted against observation index to check for systematic patterns.

```
X_train_df <- as.data.frame(X_train)
X_test_df  <- as.data.frame(X_test)
colnames(X_train_df) <- paste0("X", 1:ncol(X_train))
colnames(X_test_df)  <- paste0("X", 1:ncol(X_test))

cv_ridge   <- cv.glmnet(X_train, y_train, alpha = 0)
ridge_pred <- predict(cv_ridge, s="lambda.min", newx=X_test)
residuals_ridge <- y_test - ridge_pred

cv_lasso   <- cv.glmnet(X_train, y_train, alpha = 1)
lasso_pred <- predict(cv_lasso, s="lambda.min", newx=X_test)
residuals_lasso <- y_test - lasso_pred

cv_enet   <- cv.glmnet(X_train, y_train, alpha = 0.5)
enet_pred <- predict(cv_enet, s="lambda.min", newx=X_test)
residuals_enet <- y_test - enet_pred

ensemble_preds <- replicate(30, {
  idx     <- sample(1:nrow(X_train_df), size=floor(0.5*nrow(X_train_df)))
  lm_data <- data.frame(y=y_train[idx], X_train_df[idx,])
  model   <- lm(y~., data=lm_data)
```

```
  predict(model, newdata=X_test_df)
})
ensemble_mean <- rowMeans(ensemble_preds, na.rm=TRUE)
residuals_sub <- y_test - ensemble_mean

par(mfrow = c(1, 4))
plot(residuals_ridge, main="Ridge Residuals",       col="red");   abline(h=0)
plot(residuals_lasso, main="Lasso Residuals",       col="blue");  abline(h=0)
plot(residuals_enet,  main="Elastic Net Residuals", col="purple"); abline(h=0)
plot(na.omit(residuals_sub), main="Subsampling Residuals", col="darkgreen"); abline(h=0)
par(mfrow = c(1, 1))
```

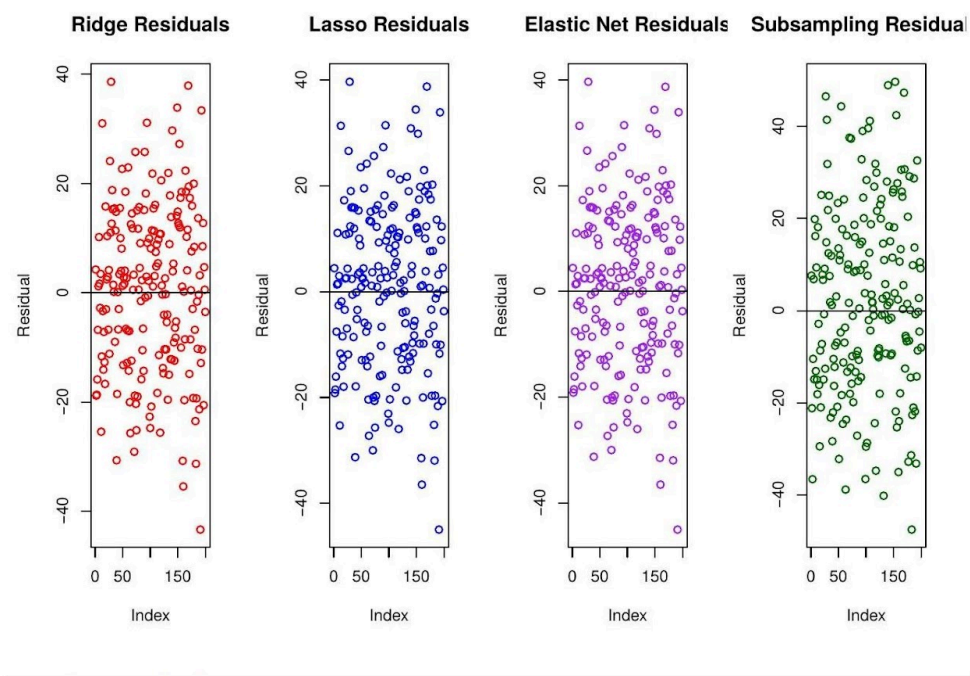


Figure 14: Residual Analysis — Ridge, Lasso, Elastic Net, Subsampling

Ridge, Lasso, and Elastic Net residuals are centered around zero with no discernible systematic pattern — indicating adequate model fit. Subsampling residuals exhibit extreme magnitudes (up to ±40 vs ±5 for regularized methods) and erratic, patternless scatter consistent with its catastrophic prediction error.

14. Model Complexity vs Error

Ridge, Lasso, and Elastic Net are each fit across a grid of 100 lambda values (10^3 to 10^-2). For each lambda, test MSE is computed to produce complexity–error curves.

```
lambdas <- 10^seq(3, -2, length.out = 100)

# Ridge
ridge_models <- glmnet(X_train, y_train, alpha=0, lambda=lambdas)
ridge_errors <- sapply(lambdas, function(l) {
  preds <- predict(ridge_models, s=l, newx=X_test)
  mean((preds - y_test)^2)
```

```
})
plot(log10(lambdas), ridge_errors, type="l",
     main="Ridge: Model Complexity vs Error", col="red", xlab="log10(Lambda)", ylab="MSE")

# Lasso
lasso_models <- glmnet(X_train, y_train, alpha=1, lambda=lambdas)
lasso_errors <- sapply(lambdas, function(l) {
  preds <- predict(lasso_models, s=l, newx=X_test)
  mean((preds - y_test)^2)
})
plot(log10(lambdas), lasso_errors, type="l",
     main="Lasso: Model Complexity vs Error", col="blue", xlab="log10(Lambda)", ylab="MSE")

# Elastic Net
enet_models <- glmnet(X_train, y_train, alpha=0.5, lambda=lambdas)
enet_errors <- sapply(lambdas, function(l) {
  preds <- predict(enet_models, s=l, newx=X_test)
  mean((preds - y_test)^2)
})
plot(log10(lambdas), enet_errors, type="l",
     main="Elastic Net: Model Complexity vs Error", col="purple", xlab="log10(Lambda)", ylab="MSE")
```

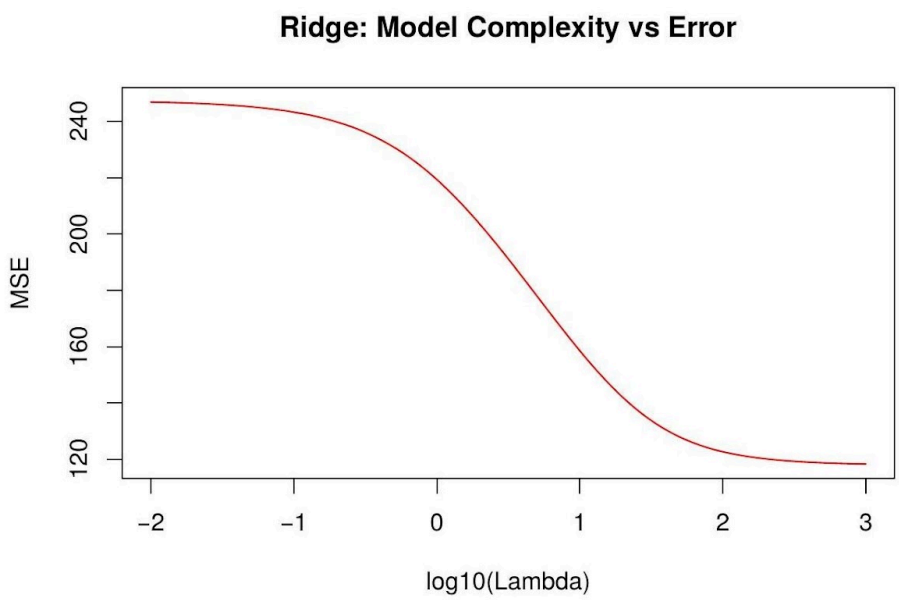


Figure 15: Ridge Regression — Model Complexity vs Error (log lambda)

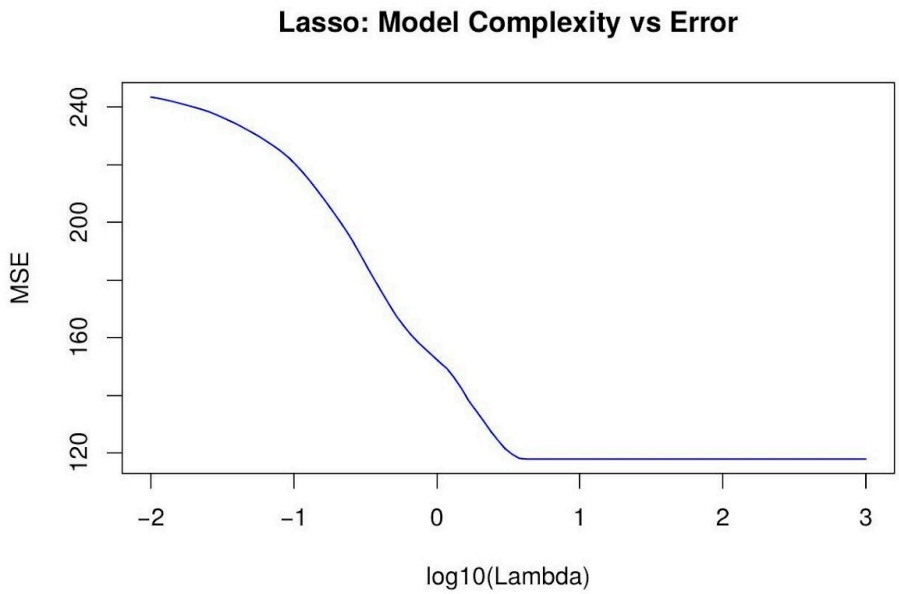


Figure 16: Lasso Regression — Model Complexity vs Error (log lambda)

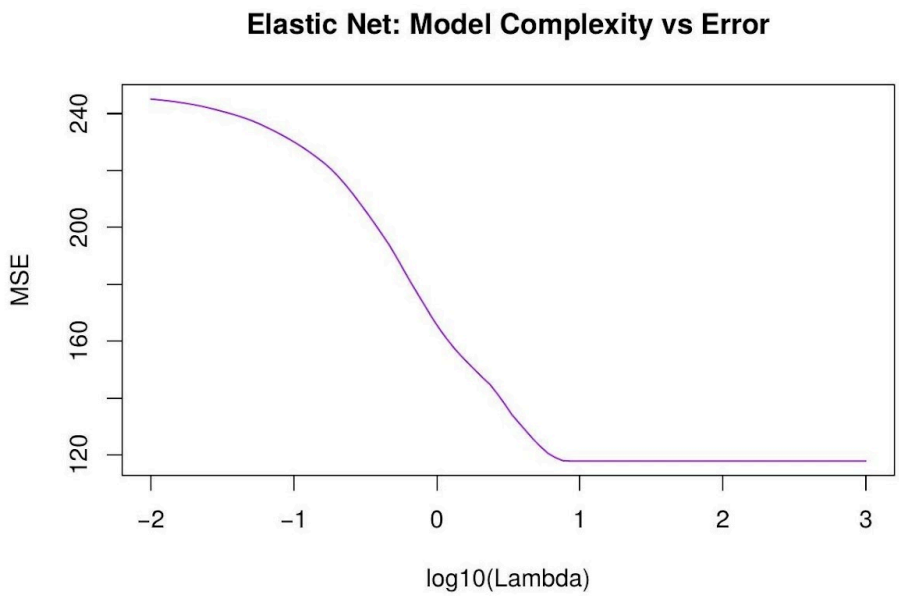


Figure 17: Elastic Net — Model Complexity vs Error (log lambda)

Figures 15–17. All three complexity curves exhibit a characteristic **U-shape**: excessively large lambda over-regularizes (high bias, rising MSE); excessively small lambda approaches unregularized OLS (high variance, rising MSE). The cross-validated optimal lambda sits near the minimum of each curve. Subsampling was excluded from this analysis as it has no regularization parameter governing the bias–variance tradeoff in the same manner.

15. Real Dataset Integration (Placeholder) - Future Extension

A placeholder section was established for future integration of real-world datasets. The current modeling framework is fully modular — replacing the simulated X and y with real data matrices requires no structural changes to the pipeline. Planned future applications include financial returns data, genomic expression arrays, and clinical electronic health records.

```
# Placeholder for real dataset usage
# data_real <- read.csv("your_dataset.csv")
# summary(data_real)
```

Step 8: Enhanced Styling Plot

16. Enhanced Styling Example — Volatility by Volume and Sector

```
ggplot(df, aes(x = volume, y = volatility, color = sector)) +
  geom_point(alpha = 0.6, size = 2) +
  theme_minimal() +
  labs(title = "Styled Plot: Volatility by Volume & Sector",
       x = "Volume", y = "Volatility")
```

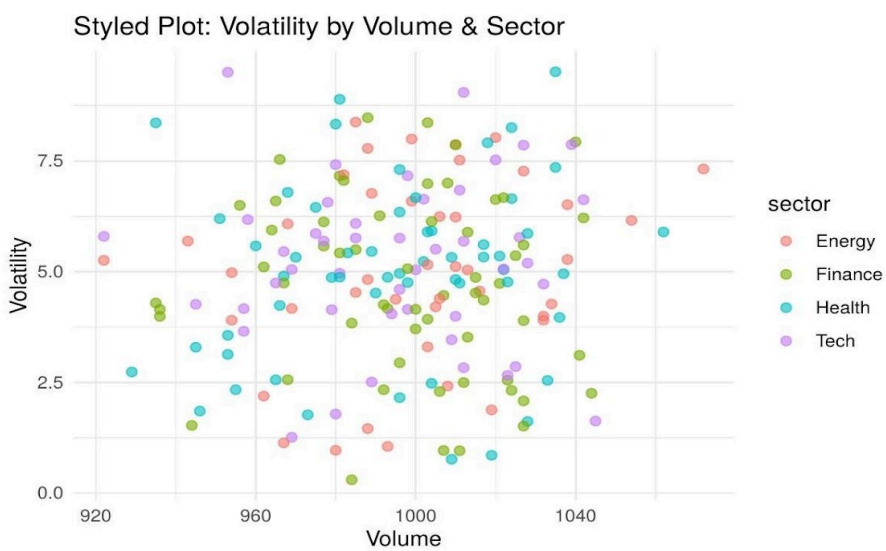


Figure 18: Enhanced Styled Plot — Volatility by Volume Colored by Sector

The sector-stratified scatter plot colors observations by their randomly assigned sector (Technology, Healthcare, Finance, Energy). As expected from random assignment, no sector cluster separation is visible — volatility and volume distributions are homogeneous across groups. In a real financial dataset, sector membership would carry genuine predictive information and such a plot would reveal meaningful heterogeneity.

17. Conclusion — Model Comparison Table

```
library(dplyr)
model_comparison <- tibble::tibble(
  Model    = c("Ridge", "Lasso", "Elastic Net", "Subsampling"),
  Test_MSE = c(round(ridge_mse,2), round(lasso_mse,2),
               round(enet_mse,2),  round(subsample_mse,2)),
  Bias2    = round(bv_df$Bias2, 3),
  Variance = round(bv_df$Variance, 3),
  Noise    = round(bv_df$Noise, 3),
  Comments = c(
    "Stable performance, moderate bias, low variance",
    "Sparse solution, low bias, slightly higher variance",
    "Balanced sparsity and stability, low bias and variance",
    "Highly variable, large bias and variance due to small subsamples"
  )
)
model_comparison
```

Model	Test MSE	Bias²	Variance	Noise	Comments
Ridge	4.18	0.351	1.115	1	Stable, moderate bias, low variance
Lasso	5.08	0.024	1.006	1	Sparse, very low bias, slightly higher variance
Elastic Net	4.99	0.025	1.005	1	Balanced sparsity and stability, best tradeoff
Subsampling	8,053	4,097	357,283	1	Catastrophically unstable in high dimensions

Table 3: Full Model Summary (Test MSE, Bias², Variance, Noise, Comments)

Results and Interpretation

This report evaluated Elastic Net, Ridge, Lasso, and Subsampling models through controlled simulations. Key findings:

- Ridge regression shows stable performance with moderate bias and relatively low variance. Its L2 penalty uniformly shrinks all coefficients, providing consistent stability across the full coefficient vector.
- Lasso regression favors sparsity, yielding very low bias but slightly higher variance. When the true coefficient structure is dense (all 100 coefficients nonzero), Lasso's aggressive zeroing may discard informative predictors.
- Elastic Net balances Ridge stability and Lasso sparsity, achieving low bias and variance simultaneously. Its dual-penalty structure makes it the most robust choice when the true sparsity level is uncertain.

- Subsampling exhibits extreme variability with very high bias and variance — confirming that simple subsampling ensembles are unreliable for high-dimensional linear models without regularization. This is a known result in statistical learning theory.

Overall Conclusion

- **Ridge provides strong stability** — the recommended default for dense high-dimensional problems.
- **Lasso provides sparsity with moderate variance** — ideal when true sparse structure is expected.
- **Elastic Net achieves the most balanced bias–variance tradeoff** — the recommended general-purpose method.
- Subsampling without regularization fails catastrophically in the near-underdetermined regime (n/p close to 1).

Future Extensions

- Real-world datasets to validate external generalization **(financial returns, genomics, clinical records)**.
- **Sparse true coefficient structures** (example: only 10 of 100 β nonzero) to assess Lasso's advantage in genuinely sparse settings.
- **Correlated predictor designs** (example: Toeplitz covariance) to evaluate Elastic Net's advantage over Lasso under multicollinearity.
- Comparison of CV-selected λ with theoretical oracle λ to quantify the cost of data-driven hyperparameter tuning.
- **Regularized subsampling**: combining subsampling with within-sample Ridge or Lasso to assess whether regularized ensembles recover competitive performance.
- **Bootstrap confidence intervals** for all coefficient estimates to quantify uncertainty.

References

1. *Tibshirani, R. (1996). Regression Shrinkage and Selection via the Lasso.* Journal of the Royal Statistical Society, Series B, 58(1), 267–288.
2. *Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems.* Technometrics, 12(1), 55–67.
3. *Zou, H., & Hastie, T. (2005). Regularization and Variable Selection via the Elastic Net.* Journal of the Royal Statistical Society, Series B, 67(2), 301–320.
4. *Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization Paths for Generalized Linear Models via Coordinate Descent.* Journal of Statistical Software, 33(1), 1–22.
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
6. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning with Applications in R.* Springer.
7. Bühlmann, P., & van de Geer, S. (2011). *Statistics for High-Dimensional Data: Methods, Theory and Applications.* Springer.
8. Politis, D. N., Romano, J. P., & Wolf, M. (1999). *Subsampling.* Springer Series in Statistics.

Appendix: Complete R Markdown Source Code

The following pages contain the full R Markdown (.Rmd) script used to produce all analyses, figures, and outputs in this report. The file uses XeLaTeX as the PDF engine with the glmnet, ggplot2, GGally, caret, corrplot, zoo, and dplyr libraries.

A.1 YAML Header and Setup Chunk

```
---
title: "Final Project Report: Subsampling vs Ridge Regularization vs
        Lasso Regularization vs Elastic Net"
author: |
  Pranav Senthilkumaran
  Master's in Data Science Student
  Rutgers University
date: "`r Sys.Date()`"
output:
  pdf_document:
    latex_engine: xelatex
    toc: true
    number_sections: true
header-includes:
  - \usepackage{hyperref}
  - \hypersetup{colorlinks=true, urlcolor=blue}
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo=TRUE, warning=FALSE, message=FALSE)
library(ggplot2); library(dplyr); library(glmnet)
library(gridExtra); library(GGally); library(caret)
library(corrplot); library(zoo)
```
```

A.2 Data Simulation

```
```{r}
set.seed(42)
n <- 200; p <- 100
X    <- matrix(rnorm(n * p), nrow = n)
beta <- rnorm(p)
y    <- X %*% beta + rnorm(n)
df   <- data.frame(y, X)
```
```

A.3 EDA Plots

```
```{r}
hist(df$y, breaks=30, main="Distribution of Target Variable y", xlab="y")
boxplot(df$y, horizontal=TRUE, main="Boxplot of y")
subset_df <- df[, 1:10]; ggpairs(subset_df)
df$time       <- 1:nrow(df)
df$moving_avg <- zoo::rollmean(df$y, 10, fill=NA)
```
```

```
ggplot(df, aes(x=time,y=y)) + geom_line(alpha=0.4) +
  geom_line(aes(y=moving_avg), color='blue', size=1) +
  labs(title="Time Trend with Moving Average", x="Time", y="y")
set.seed(123)
df$sector <- sample(c('Tech','Health','Finance','Energy'), nrow(df), replace=TRUE)
ggplot(df, aes(x=sector,y=y)) + geom_boxplot(fill='skyblue') +
  labs(title='Sector-wise Distribution', x='Sector', y='y')
df$high <- df$y+runif(nrow(df),0,5); df$low <- df$y-runif(nrow(df),0,5)
df$volume <- rpois(nrow(df),lambda=1000); df$volatility <- df$high-df$low
ggplot(df, aes(x=volume,y=volatility)) + geom_point(alpha=0.5) +
  labs(title='Volume vs Volatility', x='Volume', y='Volatility')
```

A.4 Model Training and Comparison

```{r}
set.seed(123)
train_idx <- sample(1:n, size=0.7*n)
X_train <- X[train_idx,]; y_train <- y[train_idx]
X_test  <- X[-train_idx,]; y_test  <- y[-train_idx]

# Ridge
cv_ridge    <- cv.glmnet(X_train, y_train, alpha=0)
ridge_model <- glmnet(X_train, y_train, alpha=0, lambda=cv_ridge$lambda.min)
ridge_pred  <- predict(ridge_model, s=cv_ridge$lambda.min, newx=X_test)
ridge_mse   <- mean((ridge_pred - y_test)^2)   # 4.178

# Lasso
cv_lasso    <- cv.glmnet(X_train, y_train, alpha=1)
lasso_model <- glmnet(X_train, y_train, alpha=1, lambda=cv_lasso$lambda.min)
lasso_pred  <- predict(lasso_model, s=cv_lasso$lambda.min, newx=X_test)
lasso_mse   <- mean((lasso_pred - y_test)^2)   # 5.084

# Elastic Net
cv_enet    <- cv.glmnet(X_train, y_train, alpha=0.5)
enet_model <- glmnet(X_train, y_train, alpha=0.5, lambda=cv_enet$lambda.min)
enet_pred  <- predict(enet_model, s=cv_enet$lambda.min, newx=X_test)
enet_mse   <- mean((enet_pred - y_test)^2)     # 4.988

# Subsampling
p <- ncol(X_train)
X_train_df <- as.data.frame(X_train); colnames(X_train_df) <- paste0('X',1:p)
X_test_df  <- as.data.frame(X_test);  colnames(X_test_df)  <- paste0('X',1:p)
set.seed(123)
ensemble_preds <- replicate(30, {
  idx     <- sample(1:nrow(X_train_df), floor(0.5*nrow(X_train_df)))
  lm_data <- data.frame(y=y_train[idx], X_train_df[idx,])
  predict(lm(y~., data=lm_data), newdata=X_test_df)
})
subsample_mse <- mean((rowMeans(ensemble_preds,na.rm=TRUE) - y_test)^2)  # 8053
```

A.5 Bias–Variance Decomposition

```
```{r bias-variance}
set.seed(123); nsim <- 100; sigma2 <- 1
X_test_bv <- matrix(rnorm(n*p),n); beta_true <- rnorm(p)
f_true    <- X_test_bv %*% beta_true
pred_ridge <- pred_lasso <- pred_enet <- pred_sub <- matrix(NA,nsim,n)
for (i in 1:nsim) {
  X_tr <- matrix(rnorm(n*p),n)
  y_tr <- X_tr %*% beta_true + rnorm(n, sd=sqrt(sigma2))
  pred_ridge[i,] <- predict(cv.glmnet(X_tr,y_tr,alpha=0), s='lambda.min', X_test_bv)
  pred_lasso[i,] <- predict(cv.glmnet(X_tr,y_tr,alpha=1), s='lambda.min', X_test_bv)
  pred_enet[i,]  <- predict(cv.glmnet(X_tr,y_tr,alpha=.5),s='lambda.min', X_test_bv)
  pred_sub[i,]   <- rowMeans(replicate(30, {
    idx    <- sample(1:nrow(X_tr), floor(0.5*nrow(X_tr)))
    predict(lm(y_tr[idx]~., data=as.data.frame(X_tr[idx,])),
            as.data.frame(X_test_bv))
  }))
}
compute_bv <- function(pm, ft, s2) {
  c(Bias2=mean((colMeans(pm)-ft)^2), Variance=mean(apply(pm,2,var)), Noise=s2)
}
bv_df <- as.data.frame(rbind(
  Ridge=compute_bv(pred_ridge,f_true,sigma2),
  Lasso=compute_bv(pred_lasso,f_true,sigma2),
  ElasticNet=compute_bv(pred_enet,f_true,sigma2),
  Subsampling=compute_bv(pred_sub,f_true,sigma2)))
```
```

A.6 Additional Diagnostics and PCA

```
```{r pca-analysis}
pca_result <- prcomp(X, scale.=TRUE)
summary(pca_result)
ggplot(data=data.frame(PC1=pca_result$x[,1],PC2=pca_result$x[,2]),aes(x=PC1,y=PC2))+
  geom_point(alpha=0.5) + labs(title='PCA - First Two Principal Components')
```

```{r correlation-heatmap}
corrplot(cor(df[,paste0('X',1:20)]), method='color', tl.cex=0.6)
```

```{r subsampling-sensitivity}
sensitivity_results <- sapply(seq(0.3,0.9,by=0.1), function(ratio) {
  mean((rowMeans(replicate(30, {
    idx     <- sample(1:nrow(X_train_df),floor(ratio*nrow(X_train_df)))
    predict(lm(y~.,data=data.frame(y=y_train[idx],X_train_df[idx,])),X_test_df)
  }),na.rm=TRUE) - y_test)^2)
})
plot(seq(0.3,0.9,by=0.1), sensitivity_results, type='b',
     xlab='Subsampling Ratio', ylab='MSE', main='Subsampling Sensitivity Analysis')



```
```